

1. Introduction: Welcome to the Academy

Michael Jordan was not on his high school varsity basketball team at first, and many free-throw champions also started by missing more shots than they made. They improved by learning from their mistakes, just as neural networks do. Initially, neural networks can't recognize things like cats, numbers, words, or music; they make guesses, compare those guesses with the correct answers, and then adjust to do better next time.

The report focuses on Coach Byte's Basketball Academy, which intends to use a new training robot called Bouthat that is learning to shoot. To explain how this works, the report refers to backpropagation, an algorithm that enables a network to determine what improvements are needed. First, it gives a simple example worked out by hand, then shows how to apply it in Python, and finally discusses how this method is currently used. When learners receive feedback on what to change, they can perform the task more effectively.

2. Meet the Bounce Squad

Every player on the roster stands in for a piece of the network. Keep this card handy every technical term in the rest of the report maps back to someone on this team.

Neural Network Concept	Meet the Player	What They Do
Prediction	Bounce, Shooting Guard	Takes the shot based on today's training.
Correct answer	The Hoop	The fixed target Bounce is aiming for.
Loss	Ref Distance	Measures exactly how far the shot missed by.
Backpropagation	Coach Byte	Rewinds the play in reverse to find what caused the miss.
Optimizer (Gradient Descent)	Asst. Coach Descend	Uses Coach Byte's notes to correct Bounce's form before the next shot.

3. The Five Parts of a Possession

Basketball coaches break a possession into stages; so, does a network. Here is one full “training possession,” from Bounce's first move to the correction that follows.

#	Stage	On the Court	Inside the Network
1	Forward pass	Bounce takes the shot.	Input values flow forward through the layers.
2	Prediction	The ball lands somewhere.	The network outputs its current guess.
3	Loss	How far from the hoop?	A number scores how wrong the guess was.
4	Backpropagation	Coach traces the cause backward.	The error is pushed backward, layer by layer.
5	Update	Force, angle, and aim are corrected.	Weights and biases are nudged to reduce error.

4. Under the Hood: What Happens Inside the Network

Step 1: The Forward Pass

The input values pass from the first layer to the output layer. Each connection has a weight that determines the strength of the signal it passes through, and when a neuron is active, it also adds bias. The activation function then transforms the total value into the neuron's final output, which allows the network to learn curves and patterns instead of being limited to straight lines.

Step 2: Measure Loss

A loss function turns the difference between the prediction and the actual answer into a single value; the lower the loss, the nearer the shot was to the hoop, and the higher the loss, the further it missed. The aim of training is to reduce that figure.

Step 3: Backward Pass

Backpropagation starts at the output and moves backward through each layer, using the chain rule to compute a gradient for every trainable weight and bias; each gradient tells you how a small change in that parameter would affect the loss.

Step 4: Update the Settings

An optimizer alters the parameters by means of the gradients, and the simplest version, known as gradient descent, obeys one update rule:

$$\text{new_weight} = \text{old_weight} - \text{learning_rate} * \text{gradient}$$

The value of the learning rate determines the length of each step: if it is made too large, the training process will overshoot a good solution and continue to bounce around; if it is too small, then it will have to go through a great many more practice sessions in order to achieve any improvement.

5. Backpropagation and Gradient Descent Are Different

These two terms get mixed up constantly, even by people who use them every day, so it's worth being precise. Backpropagation is the messenger: it calculates blame. Gradient descent acts on that message.

Concept	Job	Basketball Coach Analogy
Backpropagation	Calculates how each parameter affected the loss.	Determines whether force, angle, or aim caused the miss.
Gradient descent	Uses gradients to change the parameters.	Changes force, angle, and aim before the next shot.

6. Bounce's First Season: A Tiny Numerical Example

If a neuron has an input of 2, a weight of 0.5, and no bias, it multiplies the input by the weight; rather than making a single correction as in a rapid drill, we will run six trials using Bounce and note the decreasing loss.

Shot	Weight (start)	Prediction	Loss	Gradient	Weight (updated)
1	0.500	1.000	1.000000	-4.000	0.900
2	0.900	1.800	0.040000	-0.800	0.980
3	0.980	1.960	0.001600	-0.160	0.996
4	0.996	1.992	0.000064	-0.032	0.999
5	0.999	1.998	0.0000026	-0.006	0.9998
6	0.9998	1.9997	0.0000001	-0.001	0.99997

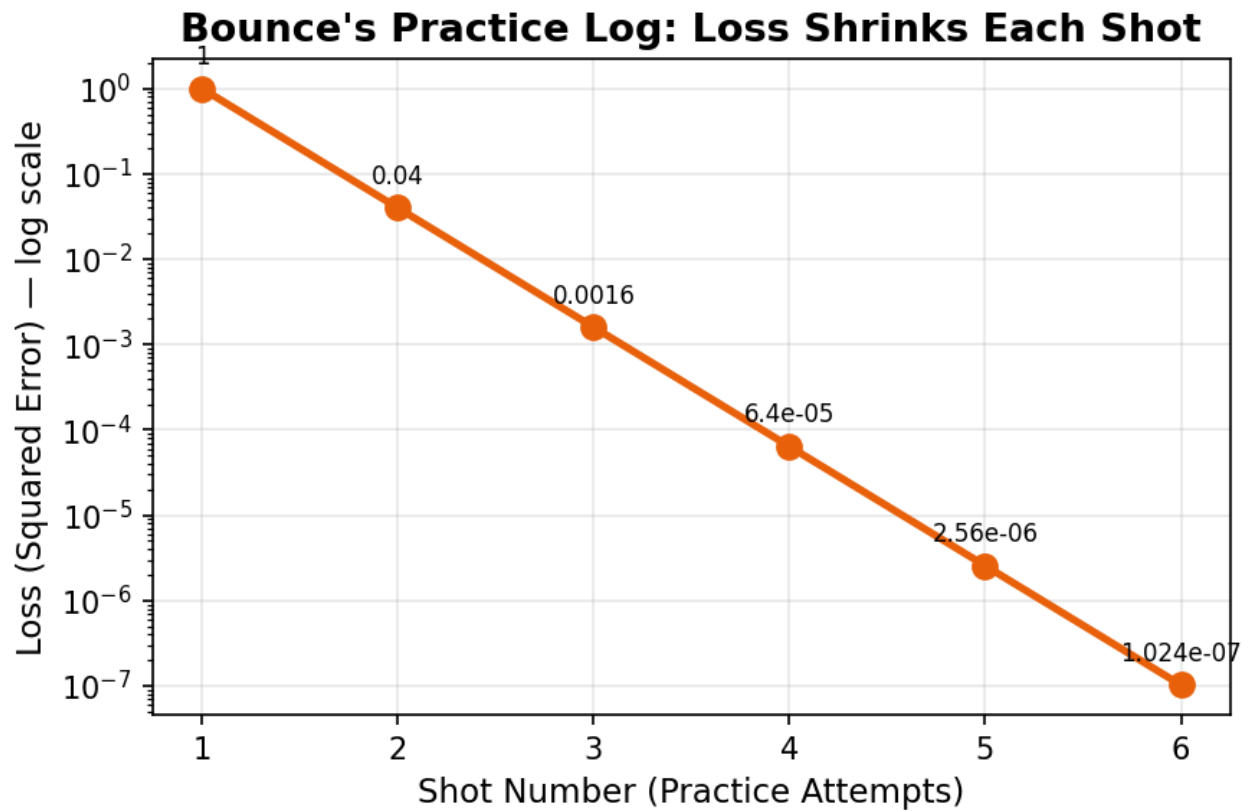


Figure 2. Loss on a log scale — each shot cuts the error by roughly 25×.

Notice the pattern: the loss does not fall by the same fixed amount each round — it shrinks by a large fraction every time. That is exactly what real training curves look like, just usually with millions of weights updating at once instead of one.

7. Show Me the Code: Bounce in Python

All the items in the table above use arithmetic, so you can run them directly on a computer. If you just want to see how they work, you can use simple Python code for the same group of single neurons, and you only need basic math functions.

Bounce's Free-Throw Training Simulator

One neuron learning to hit an exact target score

```
input_signal = 2.0  # how hard Bounce pushes off (the input)
weight = 0.5        # Bounce's current shooting form (the weight)
target = 2.0        # the hoop's exact position (correct answer)
learning_rate = 0.1 # how big a form change Coach allows per round
```

```
for shot in range(1, 7):
```

```
    prediction = input_signal * weight  # forward pass
    loss = (prediction - target) ** 2    # how far off the shot was
    gradient = 4 * (prediction - target) # backprop: blame
    weight -= learning_rate * gradient  # update: new form
```

```
    print(f"Shot {shot}: pred={prediction:.3f}, "
          f"loss={loss:.6f}, weight={weight:.4f}")
```

Frameworks like PyTorch and TensorFlow do not require you to write the gradient formula yourself. Their autograd engines create a computational graph for every operation during the forward pass and then automatically apply the chain rule on the way back. This approach lets a single idea scale from Bounce's one weight to models with billions of weights.

8. Why the Chain Rule Matters

A deep network consists of many interconnected calculations: a weight in one layer influences a neuron, which in turn affects the next, and this process continues until the result, or loss, is obtained. This is analogous to a relay race, where the total time depends on each runner. The chain rule lets backpropagation combine these individual effects into a single, definite value for each weight in one backward pass.

A Bit of History

Backpropagation was popularized in 1986 by researchers David Rumelhart, Geoffrey Hinton, and Ronald Williams, who showed that multi-layer networks could learn internal representations rather than requiring hand-programmed ones. That single insight still drives today's largest AI models.

9. Where Backpropagation Is Used Today

The network's architecture and loss functions vary by project, but the learning loop stays the same: predict, measure error, calculate gradients, then update. This is the loop currently used for training.

- Image classifiers, including tools that flag concerning patterns in medical scans
- Speech recognition systems, like the voice assistants in phones and smart speakers
- Large language models, the systems behind modern AI chatbots
- Recommendation systems that decide what shows up in a video or shopping feed
- Perception systems in robots and self-driving vehicles

10. Strengths and Limitations

Strengths	Limitations
Efficiently calculates many gradients at once	Can suffer from vanishing or exploding gradients
Works across many neural network designs	Needs data, computation, and careful tuning
Let's models improve directly from examples	May learn bias present in the training data
Learned features often transfer to new, related tasks	Requires labeled correct answers to learn from (supervised setting)

11. Glossary for Rookies

Term	Plain-English Meaning
Neural network	A system built from simple math “neurons” that learns patterns from examples.
Weight	A number that controls how strongly one signal influences the next step.
Bias	An extra adjustable number each neuron adds, giving it more flexibility.
Loss	A single number that score how wrong a prediction was.
Gradient	A signal that says which direction and how much to change a weight.
Learning rate	How big a step the optimizer takes with each update.
Epoch	One full pass through all the training examples.

12. Conclusion

The error in a neural network is propagated into actionable, useful suggestions for improvement. The process begins after the forward pass and the loss has been calculated, and it then uses the chain rule to proceed backward in such a manner that it determines the contribution of each weight and bias to the error; the optimizer then makes small and careful adjustments, just as in the example of the Bounce the basketball robot, carrying out this procedure one instance at a time until the performance has been improved.

Bounce had improved as a shooter by the end of the season as a result of having carried out a number of small and gradual adjustments, not because there was a rule requiring 'perfect form'. The real background to backpropagation is that it assigns blame to each layer, saying a layer is 'right' or 'wrong'.